Patrón de creación -> Prototype

Este patrón sirve para crear copias de un objeto base sin necesidad de instanciarlo cada vez, lo que puede generar un alto gasto de recursos en objetos con muchos atributos, entonces se genera un objeto original como “plantilla” y el resto son duplicados de dicha plantilla

Puede ser muy útil cuando se deben crear muchas unidades del mismo objeto, como por ejemplo en el caso de tener que generar muchos personajes idénticos en un juego, en vez de crear 50 personajes se crea 1(Original) y se hacen 49 copias del dicho

Tp1 > Creacion→Prototype > src > interfaces > Prototipo.java > Prototipo<T> > clonar()

```java
1  package interfaces;
2
3  public interface Prototipo<T> {
4      T clonar();
5  }
```

Tp1 > Creacion→Prototype > src > clases > PersonajeBase.java > PersonajeBase > vida

```java
1  package clases;
2
3  import interfaces.Prototipo;
4
5  import java.util.List ;
6
7  import enums.Habilidades;
8
9  import java.util.Collections;
10 //explico el uso de Collections en linea 46
11
12 public class PersonajeBase implements Prototipo<PersonajeBase> {
13
14     private String nombre;
15     private int vida;
16     private List<Habilidades> habilidades;
17
18     public PersonajeBase(String nombre, int vida, List<Habilidades> habilidades){
19         this.nombre = nombre;
20         this.vida = vida;
21         this.habilidades = habilidades;
22     }
23
24     public PersonajeBase(PersonajeBase Prototipo){
25         this.nombre = nombre;    The assignment to variable nombre has no effect
26         this.vida = vida;    The assignment to variable vida has no effect
27         this.habilidades = habilidades;    The assignment to variable habilidades has no effect
28     }
29
30     @Override
31     public PersonajeBase clonar(){
32         return new PersonajeBase(this);
33     }
34
35     @Override
36     public String toString(){
37         return "Nombre: " +nombre + ". Vida: " +vida + ". Habilidades: (" +habilidades +")";
38     }
39
40     public String getNombre(){
41         return nombre;
42     }
43
44     public int getVida(){
45         return vida;
46     }
47     public void setVida(int vida){
48         System.out.println(x: "Ingrese la vida del personaje, valor entre 1 y 100");
49         do{
50             this.vida = vida;
51         } while (vida < 1 || vida > 100);
52     }
53
54     public List<Habilidades> getHabilidades(){
55         //uso collections para poder mostrar el List como una estructura inmodificable
56         //para evitar llamados como getHabilidades.add(""), lo cual rompe integridad y seguridad
57         return Collections.unmodifiableList(habilidades);
58     }
59
60 }
```

Tp1 > Creacion→Prototype > src > clases > Barbaro.java > Barbaro

```java
package clases;

import java.util.List;

import enums.Habilidades;

public class Barbaro extends PersonajeBase{

    private int fuerza;
    private int audacia;

    public Barbaro(String nombre, int vida, List<Habilidades> habilidades, int fuerza, int audacia){
        super(nombre, vida, habilidades);
        this.fuerza = fuerza;
        this.audacia = audacia;
    }

    public Barbaro(Barbaro Prototipo){
        super(Prototipo);
        this.fuerza = fuerza;    The assignment to variable fuerza has no effect
        this.audacia = audacia;    The assignment to variable audacia has no effect
    }

    @Override
    public Barbaro clonar(){
        return new Barbaro(this);
    }

    @Override
    public String toString(){
        return super.toString() + ". Fuerza: " +fuerza +". Audacia: " +audacia;
    }

}
```

Tp1 > Creacion→Prototype > src > clases > Mago.java > Mago > toString()

```java
package clases;

import enums.Elementos;
import enums.Habilidades;

import java.util.List;

public class Mago extends PersonajeBase {

    private int mana;
    private Elementos elemento;

    public Mago (String nombre, int vida, List<Habilidades> habilidades, int mana, Elementos elemento){
        super(nombre, vida, habilidades);
        this.mana = mana;
        this.elemento = elemento;
    }

    public Mago (Mago Prototipo){
        super(Prototipo);
        this.mana = mana;    The assignment to variable mana has no effect
        this.elemento = elemento;    The assignment to variable elemento has no effect
    }

    @Override
    public Mago clonar(){
        return new Mago(this);
    }

    @Override
    public String toString(){
        return super.toString() + "Mana: " +mana + " Elemento: " +elemento;
    }

}
```

Tp1 > Creacion→Prototype > src > enums > Elementos.java > Elementos

```java
package enums;

public enum Elementos {
    AGUA,
    FUEGO,
    AIRE,
    TIERRA,
    ELECTRICIDAD
}
```

Tp1 > Creacion→Prototype > src > enums > Habilidades.java > Habilidades

```java
package enums;

public enum Habilidades {
    PARALIZAR,
    ATURDIR,
    CURAR,
    REVIVIR,
    TELETRANSPORTE,
    REPARAR
}
```

Tp1 > Creacion→Prototype > src > MainPrototipe.java > MainPrototipe > main(String[])

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

import clases.*;
import enums.*;

public class MainPrototipe {
    Run | Debug
    public static void main(String[] args){
        MainPrototipe metodos = new MainPrototipe();

        Scanner sc = new Scanner(System.in);

        int seleccion;
        int cant;
        do{
            System.out.println(x: "Bienvenido a esta prueba de prototype");
            System.out.println(x: "Ingrese que tipo de personaje quiere generar, se creara 1 original y el resto seran copias");
            System.out.println(x: "1: Magos");
            System.out.println(x: "2: Barbaros");
            seleccion = sc.nextInt();
        } while (seleccion > 2 || seleccion < 1);

        do{
            System.out.println(x: "Ingrese la cantidad a generar");
            cant = sc.nextInt();
        } while (cant < 1);

        int decision;    The value of the local variable decision is not used

            switch (seleccion) {
                case 1:
                    PersonajeBase magoBase = metodos.creacionMago(sc);
                    List<PersonajeBase> clonesGenerados = metodos.generarClones(magoBase, cant);
                    metodos.mostrarEjercito(sc, clonesGenerados);
                    break;

                case 2:
                    PersonajeBase barbaroBase = metodos.creacionBarbaro(sc);
                    List<PersonajeBase> clonesGenerados2 = metodos.generarClones(barbaroBase, cant);
                    metodos.mostrarEjercito(sc, clonesGenerados2);
                    break;
            }

    }

    public Barbaro creacionBarbaro(Scanner sc){ ···

    public Mago creacionMago(Scanner sc){ ···

    public List<Habilidades> ingresoHabilidades(Scanner sc, int cantHabilidades){ ···

    public Elementos seleccionElemento(Scanner sc){ ···

    public void mostrarEjercito(Scanner sc, List<PersonajeBase> listaClones){ ···

    public List<PersonajeBase> generarClones(PersonajeBase Prototipo, int cantCopias){ ···
}
```

Patrón de estructura -> Facade

En este patrón se simplifican los llamados a procedimientos y funciones por medio de una clase Facade, que sirve como intermediador, por ejemplo, si yo tengo 5 procedimientos relacionados en una clase, en vez de llamar uno por uno, en la clase Facade se crea un procedimiento auxiliar que es quien llama a los procedimientos mencionados anteriormente, entonces desde el archivo raíz del proyecto directamente llamamos al método definido en Facade, el cual contiene al conjunto de métodos con lógica

Sus ventajas son mejor legibilidad, capacidad de mantenimiento y escalabilidad

<u>Nota: Acá empecé a implementar el inglés, no fue hecho por ia :)</u>

Tp1 > Estructura→Facade > src > Classes > FacadeShop.java > FacadeShop

```java
package Classes;

public class FacadeShop{
    //This class is the bridge between the app brain and the customer

    Shop theShop;

    public FacadeShop(Shop sh){
        this.theShop = sh;
    }

    public void openShop(){
        theShop.openDoors();
    }

    public void makePurchase(){
        theShop.takeOrder();
    }

    public void closeShop(){
        theShop.cleanFloor();
        theShop.closeDoors();
    }

}
```

Tp1 > Estructura→Facade > src > Classes > Shop.java > Shop > cleanFloor()

```java
package Classes;

public class Shop {

    //This is where a lot of logic and functions that make the shop work should go; in this case is just a demo

    public void openDoors(){
        System.out.println(x: "Opening the shop doors");
    }

    public void takeOrder(){
        System.out.println(x: "Adding order n/a to the shopping list");
    }

    public void cleanFloor(){
        System.out.println(x: "Cleaning the floor");
    }

    public void closeDoors(){
        System.out.println(x: "Closing the doors and leaving");
    }

}
```

Tp1 > Estructura→Facade > src > MainFacade.java > MainFacade

```java
import Classes.FacadeShop;
import Classes.Shop;

public class MainFacade {
    Run | Debug
    public static void main(String[] args) throws Exception {
        Shop shop = new Shop();

        System.out.println(x: "Welcome user, Im taking your order!");

        FacadeShop candyShop = new FacadeShop(shop);
        candyShop.openShop();
        candyShop.makePurchase();
        candyShop.closeShop();
    }
}
```

Patrón de comportamiento -> Template Method

Este patrón se comporta de manera similar a una interfaz, pero más local, aquí tenemos una clase Padre que tiene un procedimiento del tipo Final para que su estructura no pueda ser modificada, donde tiene funciones ya definidas y otras funciones Abstractas que tienen en común que su funcionamiento difiere en base a la clase hija que la aplique

Se suele aplicar en programas donde se sigue una linea fija de pasos, pero con un conjunto de pasos que varían en base a las decisiones que se tomen

Tp1 > Comportamiento→Template method > src > Classes > Template.java > {} Classes

```java
package Classes;

public abstract class Template {

    public final void generateShow(){
        setStage();
        setMics();
        setInstruments();
    }

    private void setStage(){
        System.out.println(x: "Stage check!");
    }

    protected abstract void setMics();
    protected abstract void setInstruments();
}
```

Tp1 > Comportamiento→Template method > src > Classes > Trap.java > Trap

```java
package Classes;

public class Trap extends Template {

    @Override
    protected void setMics(){
        System.out.println(x: "Mics checked, autotune set to: 100%");
    }

    @Override
    protected void setInstruments(){
        System.out.println(x: "No instruments avaliable, just a synthetizer");
    }

}
```

Tp1 > Comportamiento→Template method > src > Classes > Template.java > { } Classes

```java
package Classes;

public abstract class Template {

    public final void generateShow(){
        setStage();
        setMics();
        setInstruments();
    }

    private void setStage(){
        System.out.println(x: "Stage check!");
    }

    protected abstract void setMics();
    protected abstract void setInstruments();
}
```

Tp1 > Comportamiento→Template method > src > MainTemplateMethod.java > MainTemplateMethod

```java
import Classes.*;

public class MainTemplateMethod {
    Run | Debug
    public static void main(String[] args) throws Exception {

        System.out.println(x: "Setting a trap show");
        Template trapShow = new Trap();

        trapShow.generateShow();
        System.out.println(x: "------------------");

        System.out.println(x: "Setting a metal show");
        Template metalShow =  new Metal();

        metalShow.generateShow();
        System.out.println(x: "------------------");

    }
}
```

https://github.com/AgustinEstrella/Programacion-Avanzada-2026-assignments

Agustín Estrella

Legajo: 114539